

Heizöl Vorabdisposition

Prototyp eines Avisierungsservice für geplante Heizöllieferungen. DISPO übergibt Auftragsdaten mit einem vorgesehenen Lieferzeitfenster. Der Service versendet Bestätigungsanfragen, verarbeitet Kundenreaktionen und Fristabläufe und meldet fachliche Ergebnisse an DISPO zurück. Die Lieferplanung selbst erfolgt in DISPO.

Kunden öffnen ihre persönliche Bestätigungsseite über einen Link in der Benachrichtigung. Disponenten verfolgen Touren, Aufträge und Avisierungen im Avisierungsdashboard und können dort unter anderem erneut avisieren und E-Mail-Einstellungen verwalten.

Komponenten

Bereich	Aufgabe und Technologien
Backend	Spring Boot, Java 17, Camunda 7, PostgreSQL, Flyway und REST-APIs
Frontend	React, TypeScript und Vite; Bestätigungsseite, Avisierungsdashboard und lokale DISPO-Demo
Lokaler Stack	Backend, Frontend, PostgreSQL, Mailpit, pgAdmin und DISPO Mock
Projektdokumentation	Fachliche und technische Ausarbeitung
BPMN	Modellunterlagen; die vom Backend geladenen Prozesse liegen unter backend/src/main/resources/processes

Lokal starten

Voraussetzungen

- Docker mit Docker Compose und laufender Docker Engine, unter Windows beispielsweise Docker Desktop mit Linux-Containern.
- Freie Ports **3000**, **8080**, **5432**, **1025**, **8025**, **5050** und **8090**.
- Für den ersten Build Internetzugang zum Laden der Images und Abhängigkeiten.

Der vollständige Stack wird in Containern gebaut und gestartet. Java, Maven und Node.js sind dafür auf dem Host nicht erforderlich. Für die weiter unten beschriebenen lokalen Prüfungen werden JDK 17 oder neuer sowie Node.js 24 mit npm benötigt; der Maven Wrapper ist im Repository enthalten.

1. Lokale Konfiguration anlegen

Für die lokale Präsentation wird eine Datei **backend/.env** verwendet. Docker Compose liest diese Datei und übergibt die benötigten Werte an Backend und Frontend.

`SECRET_ENCRYPTION_MASTER_KEY` dient zur Verschlüsselung gespeicherter SMTP-Passwörter; `DEV_API_KEY` ermöglicht den Dashboard-Zugang über die DISPO-Demo. Diese Werte sind für die lokale Demonstration bestimmt. Der API-Schlüssel muss zum Hash der Demo-Firma im `Dev-Seed` passen.

2. Stack bauen und starten

Aus dem Repository-Hauptverzeichnis:

```
docker compose up -d --build
docker compose ps
docker compose logs --tail=100 backend frontend
```

Vorher einen separat gestarteten Frontend-Server auf Port `3000` beenden. Das Frontend wird von Compose automatisch als eigener Container gestartet; ein separates `npm run dev` ist dafür nicht erforderlich.

Der Backend-Start einschließlich Flyway-Migrationen kann beim ersten Mal etwas dauern. Ein gestarteter Container allein bedeutet noch nicht, dass die Anwendung bereits Anfragen verarbeiten kann.

Das Backend läuft mit dem Profil `dev`. Dieses richtet lokale Dashboard-Demodaten und Mailpit-Einstellungen ein. Der DISPO Mock übernimmt lokale Statusrückmeldungen und Tracking-Anfragen. Weitere Konfiguration steht in der [Backend-Dokumentation](#).

3. Anwendung öffnen und manuell prüfen

Oberfläche / Dienst	Lokale Adresse
DISPO-Demo mit Dashboard-Zugang	localhost:3000/dispo
Swagger UI	localhost:8080/swagger-ui.html
OpenAPI-Beschreibung	localhost:8080/v3/api-docs
Mailpit: versendete E-Mails	localhost:8025
pgAdmin	localhost:5050
DISPO Mock: Statusrückmeldungen	localhost:8090/api/dispo/confirmation-status-updates
PostgreSQL / Mailpit SMTP	localhost:5432 / localhost:1025

1. Swagger öffnen und prüfen, ob die API-Beschreibung geladen wird.
2. Die DISPO-Demo öffnen und **Avisierungsdashboard öffnen** anklicken. Bei gültiger Konfiguration führt ein frischer Zugangslink in das Dashboard mit den Daten der Firma.
3. Für eine E-Mail-Prüfung im Dashboard unter **Einstellungen** eine Test-E-Mail versenden und den Eingang in Mailpit kontrollieren.

4. Für einen vollständigen Kundenablauf die unten beschriebenen Playwright-Tests ausführen. Die Dashboard-Demodaten allein lösen keine Benachrichtigungsprozesse aus.

Im Browser durchgehend `localhost` verwenden. Bei Problemen zuerst `docker compose ps` und die Backend-/Frontend-Logs prüfen. Ein abgelehnter Demo-Zugang kann auf einen fehlenden oder nicht passenden `DEV_API_KEY` hinweisen.

Wenn ein per WhatsApp empfangener lokaler Link auf dem Mac geöffnet wird, kann WhatsApp oder der Browser die Adresse automatisch von `http://...` auf `https://...` umstellen. Für die lokale Demo ohne HTTPS muss die Adresse in der Browser-Adresszeile wieder auf `http://...` geändert werden.

Falls bei einer bereits verwendeten Datenbank keine E-Mails ankommen, im Dashboard unter **Einstellungen** den SMTP-Server prüfen: Für Compose muss er `mailpit` mit Port `1025` sein. Abweichende gespeicherte SMTP-Adressen werden beim Start nicht automatisch überschrieben. Mit **Verbindung testen** lässt sich die Einstellung prüfen.

Stoppen und Änderungen übernehmen

Alle folgenden Befehle im Repository-Hauptverzeichnis ausführen:

```
docker compose stop
```

Die Datenbank bleibt im Docker-Volume erhalten. Nach Quellcodeänderungen den betroffenen Dienst neu bauen, beispielsweise mit `docker compose up -d --build frontend` oder `docker compose up -d --build backend`.

Nach einer Änderung von `DEV_API_KEY` in der verwendeten Datei genügt `docker compose up -d frontend`.

Für Frontend-Entwicklung mit lokalem Vite-Server und automatischer Aktualisierung siehe [Frontend separat starten](#).

Automatisch prüfen

Die folgenden Befehle beginnen jeweils im Repository-Hauptverzeichnis. Ein erfolgreicher Compose-Build ersetzt diese Prüfungen nicht: Das Backend-Image wird mit übersprungenen Tests gebaut.

Backend: Unit- und Integrationstests

Docker muss erreichbar sein (`docker info`). Die Integrationstests starten eigene PostgreSQL-16-Container; der Compose-Stack muss dafür nicht laufen.

```
cd backend
.\mvnw.cmd clean test
```

Unter Linux/macOS stattdessen `./mvnw clean test`, mit installiertem Maven alternativ `mvn clean test` verwenden. Ergebnisse stehen unter `backend/target/surefire-reports/`.

Falls PostgreSQL beim Teststart `invalid value for parameter "TimeZone": "Europe/Kiev"` meldet, den JVM-Zeitzonennamen für diesen Lauf explizit setzen:

```
.\mvnw.cmd "-Duser.timezone=Europe/Kyiv" clean test
```

Diesen Befehl ebenfalls im Verzeichnis `backend` ausführen.

Frontend: Typprüfung, Build und Lint

```
cd frontend
npm ci
npm run build
npm run lint
```

`build` führt die TypeScript-Prüfung und den Vite-Build aus; `lint` prüft den Code mit ESLint. Hierfür muss der Backend-Stack nicht laufen.

Browsertests mit Playwright

Zuerst den vollständigen lokalen Compose-Stack starten. Die Tests benötigen Frontend, Backend, Mailpit und DISPO Mock sowie den passenden API-Schlüssel. Der unten stehende Node.js-Aufruf liest dafür ebenfalls `backend/.env.example`; eine eigene `.env` ist nicht erforderlich. Der Compose-Parameter allein setzt keine Umgebungsvariablen für lokal gestartete Tests.

Die Suite erwartet die lokalen Mailpit-Demoeinstellungen: SMTP-Server `mailpit`, Port `1025`, keine Verschlüsselung oder SMTP-Authentifizierung, Absenderadresse `dispo@heizoel.local` und Absendername `Heizöl Disposition`. Bei einer bereits verwendeten Datenbank diese Werte vorab unter **Einstellungen** prüfen; gespeicherte eigene Absenderdaten werden vom Dev-Seed nicht zurückgesetzt.

Die Tests erzeugen Testaufträge und E-Mails und sind für die lokale Entwicklungsdatenbank vorgesehen.

```
cd frontend
npm ci
npx playwright install chromium
node --env-file=../backend/.env.example
./node_modules/@playwright/test/cli.js test
npx playwright show-report
```

Playwright prüft unter anderem Dashboard-Zugang, Auftragsansichten, E-Mail-Einstellungen, Kundenbestätigung und Tracking. Die Dienste werden von der Suite auf Erreichbarkeit geprüft, aber nicht gestartet. Details und optionale Adressüberschreibungen stehen in der [Playwright-Konfiguration](#) und den [Testhelfern](#).

Weiterführende Dokumentation und Grenzen

- [Backend: Einstieg und API-Zugänge](#)
- [Architektur, Avisierungsprozess, API](#) und [Dashboard](#)
- [Frontend: Entwicklung und Demo](#)

Der Compose-Stack ist für lokale Entwicklung und Präsentationen vorgesehen. Die DISPO-Demo verwendet eine illustrative Ansicht mit interaktivem Dashboard-Button; Demo-Seite und Demo-Zugang laufen nur im Vite-Entwicklungsserver, auch innerhalb von Docker, und gehören nicht zum statischen Frontend-Build. E-Mail wird lokal mit Mailpit geprüft. SMS und WhatsApp benötigen eine konfigurierte Twilio-Anbindung; Geocoding kann einen externen Dienst ansprechen. Der lokale DISPO Mock ersetzt keine Prüfung gegen das reale DISPO-System.